# Design and Implementation of a Zero-Dependency, Event-Driven MQTT Message Broker from First Principles

Pranav Bhat
*Dept. of Information Technology*
*NITK Surathkal*, Mangalore, India
Roll No: 231IT049
pranavbhat.231it049@nitk.edu.in

Pranav Vinodh
*Dept. of Information Technology*
*NITK Surathkal*, Mangalore, India
Roll No: 231IT050
pranavvinodh.231it050@nitk.edu.in

Rijul Raman
*Dept. of Information Technology*
*NITK Surathkal*, Mangalore, India
Roll No: 231IT056
rijulraman.231it056@nitk.edu.in

***Abstract*—The Message Queuing Telemetry Transport (MQTT) version 3.1.1 protocol is the ubiquitous publish-subscribe messaging standard for Internet of Things (IoT) and edge computing environments. However, widely used industrial message brokers (e.g., Mosquitto, EMQX, HiveMQ) depend heavily on third-party networking runtimes and extensive dependency trees, introducing non-trivial memory overhead and obscuring foundational network primitives. This paper presents the architecture, theoretical design, and Phase 1 foundation of an ultra-lightweight MQTT 3.1.1 message broker engineered from first principles in standard C (C11) with zero external networking or parsing dependencies. Operating exclusively over native Linux socket interfaces and standard POSIX system libraries, the broker resolves the classical C10K concurrency challenge via an event-driven Reactor pattern utilizing level-triggered Linux `epoll`. To handle unstructured TCP byte streams, a dynamic stream accumulation and linear compaction buffer is designed to reliably reassemble fragmented control packets under arbitrary network fragmentation. Wire-level bitwise decoders enforce OASIS MQTT 3.1.1 specification invariants, decode 1-to-4 byte variable-length integers up to 268 MB, and generate binary responses. A three-phase engineering roadmap is established, and the completed Phase 1 foundational architecture is formally detailed alongside comprehensive architectural blueprints for subsequent session persistence, Radix Tree topic routing, and Quality of Service (QoS) delivery flows.**

***Index Terms*—MQTT 3.1.1, Event-Driven Architecture, epoll Reactor, TCP Stream Defragmentation, Non-blocking I/O, Zero-Dependency Systems.**

## I. Introduction

The rapid proliferation of edge gateways, autonomous sensors, and telemetry systems has established MQTT 3.1.1 as the dominant publish-subscribe messaging standard [1]. Operating on top of reliable byte-stream transports (TCP), MQTT delivers minimal wire overhead, multi-tiered Quality of Service (QoS) delivery guarantees, and hierarchical topic namespaces.

Despite its compact wire specification, modern broker deployments are frequently monolithic. Widely deployed brokers incorporate third-party abstraction layers (e.g., `libuv`, `Boost.Asio`), complex external data structures, and heavy runtime virtual machines. This introduces notable limitations:

1) **Resource Overhead:** Large binaries and memory footprints limit deployment on resource-constrained embedded edge micro-servers.
2) **Abstraction Penalty:** High-level event libraries mask underlying transport-layer behaviors, preventing fine-tuned control over socket buffers, framing, and scheduling.
3) **Supply Chain Attack Surface:** Expansive dependency graphs increase maintenance overhead and vulnerability exposure.

To study application-layer protocol engineering from foundational principles, this project develops a **Zero-Dependency MQTT Message Broker**. Developed entirely from scratch in standard C (C11), the broker relies strictly upon the POSIX sockets API (`<sys/socket.h>`), Linux event demultiplexing (`<sys/epoll.h>`), and standard `libc`.

The project is structured across three systematic engineering phases:

- **Phase 1 (Foundation):** Native non-blocking TCP network core via Linux `epoll`, dynamic TCP stream accumulation and defragmentation buffering, bitwise OASIS MQTT 3.1.1 protocol parsing, and binary response serialization.
- **Phase 2 (Handshake & Routing Engine):** Stateful `CONNECT`/`CONNACK` handling, Clean vs. Persistent session stores, and a Topic Radix Tree (Topic Trie) supporting single-level (+) and multi-level (#) wildcards.
- **Phase 3 (Payload Fan-out & Resilience):** Multi-subscriber payload fan-out, non-blocking transmit queues, QoS 0/1 packet tracking, Keep-Alive watchdog timing ($1.5\times$ interval), and Last Will and Testament (LWT) dispatch.

This report documents the system architecture, theoretical grounding, and implementation methodology for the completed Phase 1 core, concluding with formal architectural blueprints for subsequent milestones.

Technical Report for Course Project IT365 Advanced Computer Networks, Department of Information Technology, National Institute of Technology Karnataka (NITK), Surathkal.

## II. Problem Formulation and Objectives

### A. Key Technical Challenges

Developing a compliant, high-concurrency MQTT broker without third-party frameworks presents four core challenges:

*1) Unstructured TCP Streams and Defragmentation:* TCP provides a reliable, ordered, but unstructured byte stream that does not preserve application-layer packet boundaries [2]. Transport-layer segmentation (MTU boundaries, socket buffers, and Nagle's algorithm) causes packets to arrive fragmented across multiple reads (even 1 byte at a time) or concatenated in single chunks. The broker must frame variable-length packets dynamically without memory exhaustion.

*2) Wire-Level Bitwise Protocol Decoding:* OASIS MQTT 3.1.1 enforces strict binary layouts [1]. The 1-byte Fixed Header combines a 4-bit packet type with 4-bit operational flags. Spec Table 2.2 mandates strict reserved bit values (e.g., `CONNECT`, `PINGREQ`, and `DISCONNECT` must have flags `0000`; `SUBSCRIBE` must have `0010`). Furthermore, the Remaining Length field uses a variable 7-bit continuation encoding across 1 to 4 bytes (supporting sizes up to 268 MB). Any non-conformance requires immediate socket termination.

*3) High-Throughput Concurrency (The C10K Problem):* Traditional thread-per-connection architectures allocate dedicated POSIX threads per socket. Under high concurrency ($C \geq 10{,}000$), thread stacks and kernel context-switching induce catastrophic CPU and memory degradation [4]. An asynchronous, non-blocking single-reactor architecture is required.

*4) Hierarchical Wildcard Topic Resolution:* MQTT features forward-slash-delimited topic trees with wildcards ('+' matching single levels, '#' matching multiple subordinate levels). Standard hash maps cannot evaluate wildcards, while linear subscription scans scale poorly as $O(N)$ with subscriber volume [6]. A prefix tree is needed to guarantee $O(K)$ lookup, where $K$ is the path depth.

### B. System Design Objectives

- **Objective 1 (Native Non-Blocking Network Core):** Linux `epoll` reactor handling concurrent sockets with non-blocking I/O.
- **Objective 2 (Stream Defragmentation Buffer):** Dynamic linear buffer (`byte_buffer_t`) featuring geometric resizing and $O(M)$ compaction to frame variable-length packets safely.
- **Objective 3 (Bitwise Protocol Engine):** Wire-level decoder enforcing OASIS Table 2.2 flag rules and variable integer parsing.
- **Objective 4 (Radix Tree Topic Engine):** Topic Trie providing $O(K)$ wildcard resolution for + and # subscriptions.
- **Objective 5 (Stateful Session Management):** Persistent session tracking, QoS 0/1 packet flows, and keep-alive watchdog timing.

## III. Literature Survey and Theoretical Foundations

The architecture synthesizes core principles from canonical literature:

### A. OASIS MQTT Version 3.1.1 Standard

Banks and Gupta [1] specify the binary wire format and operational invariants of MQTT 3.1.1. Control packet types, variable-byte remaining lengths, and QoS handshakes are strictly governed by spec rules that require immediate session disconnection on reserved flag violations.

### B. Event-Driven Concurrency and the C10K Challenge

Kegel [4] demonstrated that operating system scaling is fundamentally governed by I/O multiplexing primitives. While legacy `select()` and `poll()` require $O(N)$ descriptor scans per event, Linux `epoll` utilizes a kernel Red-Black tree and an active poll queue, providing $O(1)$ event notifications relative to total monitored descriptors [5].

### C. UNIX Network Programming Primitives

Stevens et al. [2] formulate the core semantics of non-blocking socket programming. Applying `O_NONBLOCK` prevents thread stall on empty socket queues. Disabling Nagle's algorithm via `TCP_NODELAY` eliminates 40 ms delayed-ACK latencies on small 2–4 byte MQTT control packets. Sockets are configured with `SO_REUSEADDR` and `SO_REUSEPORT` for instant restart capability, and `SIGPIPE` is masked to prevent process crashes on premature client drops.

### D. Topic Routing via Prefix Trees

Morrison [3] introduced PATRICIA trees, demonstrating that multi-way prefix trees reduce search complexity for string keys. Jiang et al. [6] showed that tokenizing MQTT topic levels on ''/'' restricts traversal depth strictly to the topic level count $K$, eliminating dependency on subscriber count $N$.

## IV. System Architecture and Methodology

### A. Native Non-Blocking Network Core (`src/server.c`)

The network core is built entirely on native Linux primitives:

- **Master Socket Setup:** Created via `socket(AF_INET, SOCK_STREAM, 0)`. Configured with `SO_REUSEADDR` and `SO_REUSEPORT` for instant rebinding, set non-blocking with `fcntl(O_NONBLOCK)`, bound to `0.0.0.0:1883`, and marked for passive listening.
- **Linux epoll Demultiplexing:** An epoll instance is created via `epoll_create1(EPOLL_CLOEXEC)`. The master socket is monitored for `EPOLLIN`. An `accept4(..., SOCK_NONBLOCK)` loop drains pending connection bursts until `EAGAIN` is reached.
- **Socket Optimization & Safety:** Accepted sockets immediately have `TCP_NODELAY` set to disable Nagle's algorithm. Sockets are registered with `EPOLLIN | EPOLLRDHUP | EPOLLERR`. Broken pipe crashes are averted by executing `signal(SIGPIPE, SIG_IGN)`.

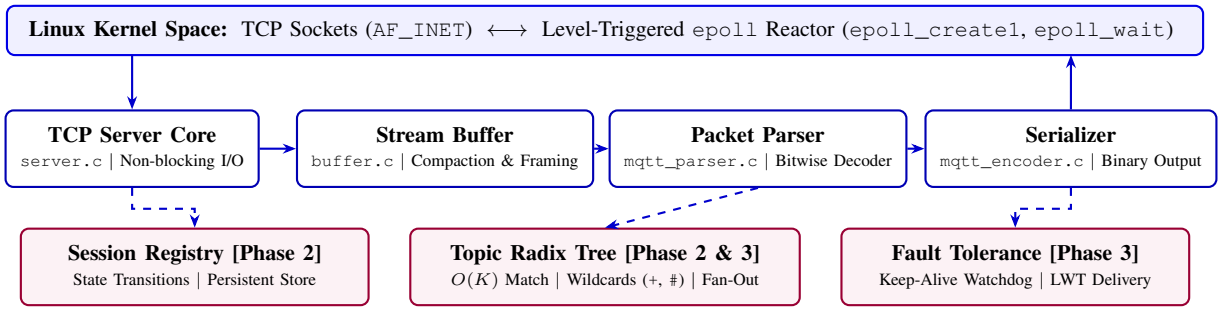



Fig. 1. System Architecture of the Zero-Dependency MQTT Broker: Kernel Event Reactor, Stream Framing, Protocol Decoding, and Topic Routing Engine.

### *B. TCP Stream Accumulation and Compaction Buffer (`src/buffer.c`)*

TCP stream framing is managed by a linear accumulation buffer:

$$\text{buffer} = \{\text{data}, \text{size}, \text{capacity}, M_{\max}\} \tag{1}$$

- **Geometric Growth:** Memory capacity doubles dynamically up to a strict bounded safety ceiling ($M_{\max} = 512\,\text{KB}$). Writes exceeding this ceiling return `MQTT_ERR_BUFFER_OVERFLOW`, terminating the connection to protect against memory exhaustion attacks.
- **Linear Compaction:** When a packet of size $P$ is consumed, `byte_buffer_drain()` invokes `memmove()` to slide remaining bytes to offset 0:

$$\text{offset } 0 \longleftarrow \text{data}[P \ldots N] \tag{2}$$

- **Zero Truncation:** If incoming data is incomplete, the parser returns `MQTT_NEED_MORE_DATA` without consuming bytes, awaiting subsequent TCP reads.

TABLE I
THREE-PHASE ENGINEERING ROADMAP

| Phase | Core Deliverables | Status |
|---|---|---|
| **Phase 1: Foundation** | Non-blocking TCP server (`epoll`), stream buffer (`byte_buffer_t`), bitwise MQTT 3.1.1 parser, encoders, and automated test suite. | **Completed** |
| **Phase 2: Handshake & Routing** | `CONNECT`/`CONNACK` state machine, Clean vs Persistent sessions, Topic Radix Tree (Trie) with + and # wildcards, and `SUBSCRIBE`/`SUBACK`. | **Current Focus** |
| **Phase 3: Fan-Out & Resilience** | PUBLISH payload fan-out, non-blocking TX queue, QoS 0/1 packet tracking, Keep-Alive watchdog (1.5×), and Last Will (LWT) delivery. | **Planned** |

### *C. Bitwise Wire-Level MQTT 3.1.1 Decoder (`src/mqtt_parser.c`)*

- **Fixed Header Parsing:** Byte 0 extracts packet type ($\text{Byte}_0 \gg 4$) $\wedge$ 0x0F and flags $\text{Byte}_0 \wedge$ 0x0F. Flags are strictly validated against OASIS Table 2.2: types `CONNECT`, `PINGREQ`, and `DISCONNECT` strictly require Flags $= 0$; types `SUBSCRIBE`, `PUBREL`, and `UNSUBSCRIBE` require Flags $= 2$; and `PUBLISH` admits valid DUP, QoS (0–2), and RETAIN combinations. Invalid combinations (e.g., QoS 3, or QoS 0 with DUP $= 1$) trigger immediate socket termination.
- **Variable-Byte Remaining Length:** Evaluates bytes $b_0 \ldots b_m$ ($m \leq 3$):

$$R = \sum_{i=0}^{m} (b_i \wedge \text{0x7F}) \times 128^i \tag{3}$$

  Encodes lengths up to 268 MB. A continuation bit set on byte 4 yields `MQTT_ERR_MALFORMED`.
- **Implemented Packet Decoders:** Full decoders for `CONNECT` (protocol name "MQTT", level 0x04, CleanSession, KeepAlive, ClientID, credentials), `PUBLISH` (validating topic has no wildcards per MQTT-4.7.1-1, PacketID, payload), `SUBSCRIBE` (PacketID, topic filters, requested QoS), `PINGREQ`, and `DISCONNECT`.

### *D. Binary Response Serialization Engine (`src/mqtt_encoder.c`)*

Direct binary wire encoding is implemented for broker responses:

- **CONNACK:** 4-byte wire frame [0x20, 0x02, SessionPresent, ReturnCode].
- **PINGRESP:** 2-byte heartbeat response [0xD0, 0x00].
- **PUBACK:** 4-byte acknowledgement [0x40, 0x02, $\text{PacketID}_{\text{hi}}$, $\text{PacketID}_{\text{lo}}$].
- **SUBACK:** Constructs byte `0x90`, encodes variable length, 16-bit PacketID, and granted QoS return codes.

### *E. Architectural Blueprint for Phases 2 and 3*

- **Topic Radix Tree (Topic Trie):** Multi-way prefix tree where nodes represent topic segments split by ''/''. Child branches handle exact string matches and wildcards ('+' for single levels, '#' for subordinate suffix levels). Topic lookup operates in $O(K)$ time, strictly bounded by topic depth $K$ [6].
- **Session State Machine:** Implements state transitions NEW $\rightarrow$ CONNECTED $\rightarrow$ CLOSED. Duplicate ClientIDs disconnect older sockets per MQTT-3.1.4-2. Persistent sessions (`clean_session == 0`) retain subscription maps and an offline QoS 1 queue across client reconnections.

- **Publish Fan-out & Deduplication:** On `PUBLISH`, the Trie is traversed to match subscribers. Per MQTT-3.3.5-1, a deduplication hash set ensures clients with overlapping wildcard subscriptions receive payloads exactly once at maximum granted QoS.
- **Fault Tolerance & Keep-Alive Watchdog:** Client active timestamps are tracked. If silent for $\Delta t > 1.5 \times$KeepAlive, the broker forcibly severs the connection. Upon ungraceful drops, stored Last Will and Testament (LWT) messages are automatically published to peer subscribers.

### *F. Phase 1 Verification Methodology*

The completed Phase 1 implementation was validated using a rigorous three-tier automated testing pyramid:

1) **Buffer Unit Tests (`test_buffer.c`):** Validates dynamic capacity doubling, memory ceiling enforcement (512 KB ceiling), and linear compaction sliding unconsumed bytes without memory leaks.
2) **Protocol Parser Tests (`test_parser.c`):** Verifies variable-length boundary conditions across 1, 2, 3, and 4-byte boundaries (0 to 268,435,455), OASIS Table 2.2 reserved flag violation rejection, and multi-packet stream pipelining.
3) **End-to-End Integration Suite (`test_server_e2e.py`):** Automates broker daemon lifecycle management via pure Python sockets. Tests full `CONNECT` $\rightarrow$ `CONNACK` handshakes, `PINGREQ` $\rightarrow$ `PINGRESP` heartbeats, QoS 1 `PUBACK` delivery, and concurrent handling across 30 simultaneous client sockets in parallel. Crucially, stream defragmentation is stressed by sending packets byte-by-byte with artificial delays, confirming the broker accumulates and reassembles stream segments flawlessly.

Table II summarizes the Phase 1 test coverage across all layers.

TABLE II
PHASE 1 AUTOMATED TEST VERIFICATION MATRIX

| Test Suite | Target Verification Scope | Pass Rate |
|---|---|---|
| `test_buffer.c` | Growth, compaction, safety limits | 100% (2/2) |
| `test_parser.c` | Framing, Table 2.2 flags, variable ints | 100% (7/7) |
| `test_server_e2e.py` | Handshakes, heartbeats, 30 clients | 100% (6/6) |

## REFERENCES


[1] A. Banks and R. Gupta, "MQTT Version 3.1.1," OASIS Standard, Oct. 2014. [Online]. Available: http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

[2] W. R. Stevens, B. Fenner, and A. M. Rudoff, *UNIX Network Programming: The Sockets Networking API*, vol. 1, 3rd ed. Boston, MA, USA: Addison-Wesley, 2003.

[3] D. R. Morrison, "PATRICIA—Practical Algorithm To Retrieve Information Coded in Alphanumeric," *Journal of the ACM*, vol. 15, no. 4, pp. 514–534, Oct. 1968.

[4] D. Kegel, "The C10K problem," 1999. [Online]. Available: http://www.kegel.com/c10k.html

[5] D. C. Schmidt, M. Stal, H. Rohnert, and F. Buschmann, *Pattern-Oriented Software Architecture: Patterns for Concurrent and Networked Objects*, vol. 2. New York, NY: John Wiley & Sons, 2000.

[6] H. Jiang, F. Shen, S. Chen, M. Du, and N. Xi, "Lightweight and High-Performance Topic Routing Algorithms for MQTT," *IEEE Internet of Things Journal*, vol. 4, no. 6, pp. 1999–2008, Dec. 2017.